

responsabilità specifica, per garantire l'isolamento dei processi e la scalabilità indipendente dei componenti.

Tutti i container comunicano tra di loro tramite una rete bridge interna dedicata (definita nel `docker-compose.yml`). I database e i broker di Messaggistica non sono accessibili direttamente dall'esterno, ma solo dai microservizi e dall'host locale per scopi di manutenzione, aumentando la sicurezza dell'architettura.

homework_2	-	-	-	✖ ✖	■	:
mailhog	AMD64	c5699c28f176	mailhog/m	1025:1025	✖ ✖	■
kafka		d161eacfa271	confluentinc/kafka	29092:29092	✖ ✖	■
data-db	AMD64	34cb8f656c8b	mysql:8.0	3308:3308	✖ ✖	■
hw_2-user-db		3a4446ae60e3	mysql:8.0	3307:3307	✖ ✖	■
api-gateway		4573bec363bf	nginx:latest	80:80	✖ ✖	■
user-manager		83123743d724	homework	5001:5001	✖ ✖	■
hm2-notifier-syster		d2a6478c22aa	homework		✖ ✖	■
hm-alert-system		0f0b9f6285a0	homework		✖ ✖	■
data-collector		05933ad9b407	homework	5002:5002	✖ ✖	■

SCELTE PROGETTUALI

Per lo sviluppo del sistema è stato utilizzato il linguaggio Python, supportato dal framework Flask. Questa scelta è motivata dall'adattabilità di Python nella gestione dei dati e dall'ampia disponibilità di librerie specializzate per i sistemi distribuiti.

Per garantire il disaccoppiamento è stato scelto di utilizzare un database MySQL per ogni microservizio. Questo garantisce che un eventuale carico eccessivo sui dati dei voli non impatti le prestazioni della gestione utenti.

L'uso di *Docker* assicura che l'ambiente di esecuzione sia isolato e riproducibile su qualsiasi macchina, risolvendo le dipendenze software.

Nella registrazione utenti è stata implementata la logica *At-Most-Once*, basata su token di idempotenza. Il client invia, insieme ai dati, un identificativo univoco della richiesta (`request_id`). Lo *User Manager* verifica in una tabella di log dedicata (`request_log`) se quell'ID è già stato processato. Se l'ID è presente, il server restituisce la risposta salvata in precedenza senza rieseguire l'inserimento. Se l'ID è nuovo, il server procede con la creazione dell'utente. Questo meccanismo garantisce che l'operazione di registrazione venga eseguita esattamente una volta (o nessuna), anche in

caso di ritrasmissioni multiple dovute a errori di rete.

Per l'Homework 2, il sistema è evoluto verso un modello a eventi. È stato introdotto Apache Kafka per gestire la comunicazione asincrona tra il Data Collector, l'Alert System e il Notifier. Questa scelta permette di disaccoppiare completamente la fase di raccolta dati dalla fase di notifica. Il Data Collector può scaricare i voli senza dover attendere i tempi di latenza dell'invio email o dell'analisi, migliorando drasticamente le performance generali.

Considerando la dipendenza critica da un servizio esterno (OpenSky Network), è stato implementato il pattern *Circuit Breaker* (tramite la libreria *pybreaker*). Se le chiamate all'API esterna falliscono consecutivamente per un numero predefinito di volte il circuito si apre e il sistema smette temporaneamente di inviare richieste per un periodo. Questo previene il sovraccarico del sistema e permette il recupero del servizio esterno.

COMUNICAZIONE E INTERAZIONE

Il sistema utilizza un approccio di comunicazione ibrida per ottimizzare le prestazioni:

- *REST (HTTP/JSON)*: utilizzato per l'interazione tra Client esterno (Postman) e i microservizi.
- *gRPC*: utilizzato per la comunicazione interna sincrona tra Data Collector e User Manager.
- *Messaggistica Asincrona*: utilizzato per il flusso dei dati di volo e allarmi. Il Data Collector pubblica i dati di volo sul topic `flight_data`, mentre l'Alert System pubblica gli allarmi sul topic `alerts`, garantendo che l'invio delle email non blocchi il processo di scaricamento dati.

IMPLEMENTAZIONE DELLA POLITICA "At-Most-Once"

Per soddisfare il requisito nella registrazione utenti, è stata implementata una politica *At-Most-Once*.

In caso di errore di rete, un client potrebbe inviare nuovamente la stessa richiesta di registrazione, rischiando di creare duplicati o errori non gestiti.

La soluzione implementata consiste che il client invia, insieme ai dati, un identificatore univoco della richiesta (`request_id`), lo *User Manager* mantiene una tabella dedicata `request_log` nel database. All'arrivo di una richiesta, il servizio verifica se quel `request_id` è già stato processato; se è stato già processato restituisce immediatamente la risposta salvata in

precedenza, senza rieseguire l'inserimento, altrimenti esegue l'inserimento dell'utente e salva l'esito nel log. Questa operazione garantisce che l'utente venga creato al massimo una volta, indipendentemente dal numero di tentativi del client.

LISTA DELLE API IMPLEMENTATE

Il sistema espone le seguenti interfacce RESTful:

- Microservizio User Manager (Porta 5001)
POST/register: Registra un nuovo utente. Richiede: email, first_name, last_name, request_id.
- Microservizio Data Collector (Porta 5002)
POST/add_interest: Aggiunge un aeroporto da monitorare. Richiede: email, airport_code, high_value (soglia traffico alto), low_value (soglia traffico basso).
GET/stats/last/<code>: restituisce i dati dell'ultimo volo registrato per un dato aeroporto.
GET/stats/average/<code>/<days>: Calcola e restituisce la media dei voli in arrivo negli ultimi X giorni.

DETTAGLIO IMPLEMENTATIVO DEI NUOVI MICROSERVIZI

Data Collector: oltre alle funzioni base, ora agisce come Producer. Una volta scaricati i voli, il servizio estrae il numero totale di arrivi e pubblica un messaggio JSON sul topic flight_data ({ "airport_code": "JFK", "arrivals": 92 }).

Alert System: è il cuore della logica di controllo. Agisce come Kafka Consumer ascoltando il topic flight_data. Per ogni messaggio ricevuto:

- Interroga il database data_db (tabella interests) per trovare utenti interessati a quell'aeroporto.
- Confronta il numero di voli con le soglie high_value e low_value definite dall'utente.
- Se una soglia è violata, produce un messaggio di allerta sul topic alerts.

Notifier System: Ascolta il topic alerts. Quando riceve un messaggio, utilizza la libreria smtp lib per costruire un'email formattata e inviarla al server SMTP locale (MailHog).

TEST E VALIDAZIONE DEL SISTEMA

La validazione del sistema è stata effettuata tramite test eseguiti con Postman e verifica diretta sui log dei container e su MailLog. Di

Di seguito troviamo tre scenari di test principali.

1. Test di Registrazione Utente

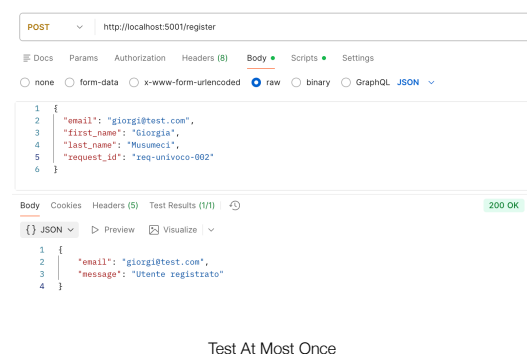
Verifica la corretta esposizione dell'API REST dello User Manager e la persistenza dei dati nel database user_db.

È stata inviata una richiesta POST all'endpoint /register contenente i dati anagrafici di un nuovo utente e un request_id univoco. Il server ha risposto con codice 201 Created, confermando l'avvenuto inserimento nel database.



2. Test della politica At-Most-Once

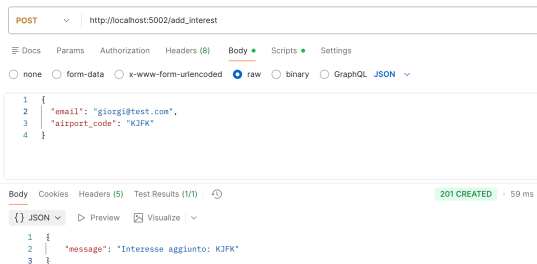
Dimostra la robustezza del sistema in caso di richieste duplicate. È stata inviata nuovamente la stessa identica richiesta del test precedente, mantenendo invariato il request_id. Il sistema non ha generato errori di chiave duplicata nel database. Al contrario, ha riconosciuto l'ID della richiesta tramite la tabella di log interna e ha restituito la risposta memorizzata precedente. Questo conferma che l'operazione di registrazione viene eseguita al massimo una volta, garantendo la consistenza dei dati.



3. Test di comunicazione gRPC

Verifica la comunicazione distribuita tra i microservizi. È stata inviata una richiesta POST all'endpoint /add_interest del Data Collector, specificando l'email dell'utente appena creato e un codice aeroportuale. Per soddisfare la

richiesta, il Data Collector ha dovuto sospendere l'elaborazione e contattare lo user Manager tramite protocollo gRPC per verificare l'esistenza dell'email. Il server ha risposto con *201 Created*, questo dimostra che il canale gRPC tra i container è attivo e funzionante. I test effettuati confermano che l'architettura a microservizi è stata implementata correttamente.



Test di comunicazione gRPC

4. Test del Flusso di Alerting e Notifica

Verifica l'intera catena Event-Driven. È stato simulato l'arrivo di dati di volo (es. 92 voli su KJFK) che violavano la soglia minima imposta dall'utente (low_value:5000).

#	id	airport_code	query_time	arrivals_co	departures_co
50	50	KJFK	2025-12-19 08:48:31	92	8

#	id	email	airport_code	high_val	low_value
1	1	giorgi@test.com	LIRF	100	10
2	2	giorgi@test.com	LIMC	100	10
3	3	giorgi@test.com	EGLL	100	10
4	5	giorgi@test.com	KJFK	10000	5000

Analizzando i log, il Data Collector ha inviato il dato a Kafka.

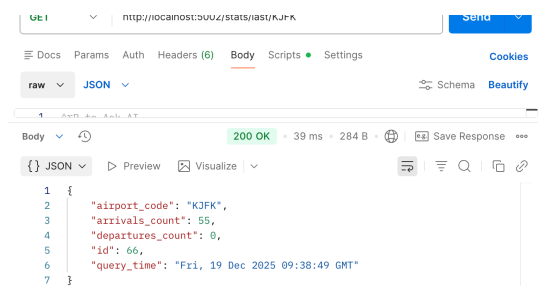
From	Subject	Time	Size
alertsystem@kafka.com	ALERT YELLOW TRAFFIC su KJFK	6 hours ago	340 B
giorgi@test.com	ALERT YELLOW TRAFFIC su KJFK	6 hours ago	340 B
giorgi@test.com	ALERT YELLOW TRAFFIC su KJFK	6 hours ago	340 B

L'Alert System ha rilevato la violazione (poiché $92 < 5000$) e generato un allarme di tipo LOW_TRAFFIC. Per la verifica finale, accedendo all'interfaccia di MailHog (localhost:8025), è stata confermata la ricezione dell'email di notifica con i dettagli corretti dell'allarme.

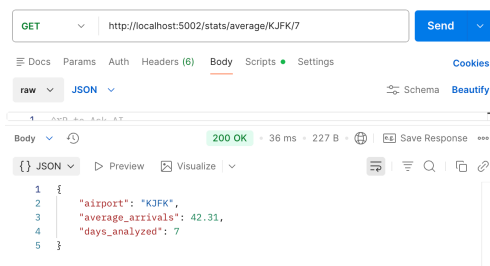
5. Test delle API di Statistica e Storico

Infine, è stata verificata la funzionalità di consultazione dello storico dei voli, per garantire che i dati scaricati dallo Scheduler vengano correttamente mantenuti e siano interrogabili.

Ultimo Volo: È stata inviata una richiesta GET all'endpoint /stats/last/KJFK. Come mostrato in figura, il sistema ha restituito un oggetto JSON contenente i dettagli dell'ultimo scaricamento (55 arrivi registrati il 19 Dicembre), confermando l'avvenuta scrittura nel database.



Calcolo Media: È stata inviata una richiesta GET all'endpoint /stats/average/KJFK/7. Il sistema ha interrogato lo storico ed elaborato i dati, restituendo una media di 42.31 arrivi negli ultimi 7 giorni.



INTERAZIONI

Di seguito è riportato il diagramma di sequenza che mostra due flussi principali: interazione con l'utente (aggiunta dell'interesse e controllo gRPC), Scheduler e Alerting.

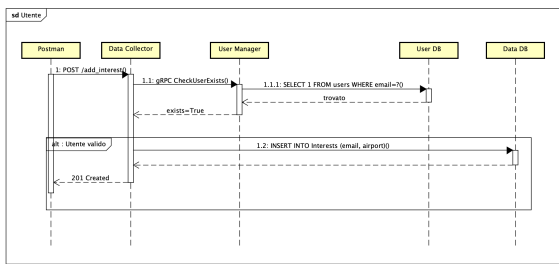


Diagramma di sequenza: Aggiunta interesse

In questo diagramma si mostra il flusso di registrazione di un interesse. Il Data Colelctor riceve una richiesta REST e sospende l'esecuzione per effettuare una chiamata sincrona gRPC verso lo User Manager. Lo User Manager interroga lo User DB per validare l'identità dell'utente e restituisce l'esito.

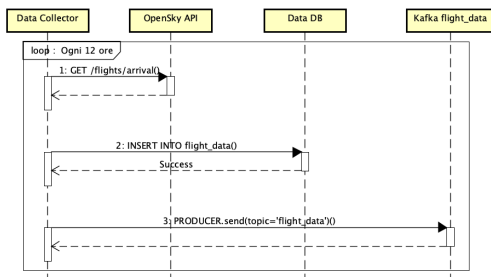


Diagramma di Sequenza: Scheduler e Data Collection

In questo diagramma lo Scheduler interno attiva ciclicamente il processo di aggiornamento, interrogando l'API esterna di OpenSky. I dati scaricati vengono salvati nel Data DB per le analisi statistiche e, contestualmente, vengono pubblicati sul topic 'flight_data' di Kafka per attivare la catena di allarmi.

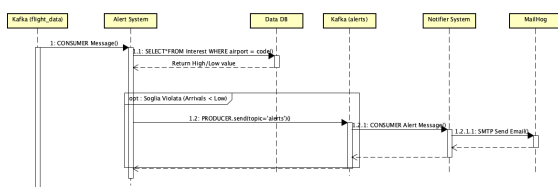


Diagramma di Sequenza: Alerting e Notifica

Infine, questo diagramma mostra il flusso asincrono Event-Drive. L'Alert System consuma i dati dal topic 'flight_data', verifica le regole presenti nel DB e in caso di violazione, produce un evento su 'alerts'. Il Notifier System intercetta l'evento e invia la notifica via email.

CONCLUSIONI

Il progetto ha raggiunto pienamente gli obiettivi prefissati, realizzando un'architettura a microservizi distribuita, scalabile e robusta. L'implementazione ha rispettato i requisiti, dimostrando la corretta applicazione dei pattern architetturali tipici dei sistemi distribuiti. L'evoluzione verso un'architettura Event-Driven con Kafka ha garantito il disaccoppiamento tra chi genera i dati (Collector) e chi li elabora (Alert/Notifier), rendendo il sistema altamente scalabile. L'introduzione del Circuit Breaker ha aggiunto robustezza contro i fallimenti dei servizi esterni.